



Take-Home Assignment: Cinema Ticket Booking System

วัตถุประสงค์

โจทย์นี้ออกแบบมาเพื่อประเมินความสามารถของผู้สมัครในด้าน

- Backend / Frontend Engineering
- Concurrency & Distributed System
- System Design & DevOps
- คุณภาพโค้ดและการอธิบายแนวคิด

เปรียบเทียบเหมือนการสร้าง “ตู้ขายตั๋วหนังออนไลน์”
ไม่ใช่แค่รันได้ แต่ต้อง **ไม่พังเมื่อมีคนแย่งกันซื้อ**

🕒 ระยะเวลา

3 วัน (72 ชั่วโมง) นับจากเวลาที่ผู้สมัครเริ่มทำงาน

1. Tech Stack (บังคับใช้)

Layer	Technology
Backend	Go (แนะนำ Gin / Echo)
Frontend	Vue.js (Vue 3 แนะนำ)

Database	MongoDB
Cache / Lock	Redis (ใช้ทำ Distributed Lock เท่านั้น ไม่ใช่แค่ cache)
Realtime	WebSocket (หรือ SSE)
Message Queue	Kafka / RabbitMQ / Redis Pub-Sub (เลือก 1)
Auth	Google OAuth 2.0 หรือ Firebase Auth
Deployment	Docker + docker-compose.yml

! ต้องสามารถรันทั้งระบบได้ด้วยคำสั่งเดียว

None

```
docker compose up --build
```

2. Functional Requirements

2.1 User Side (ผู้ใช้งานทั่วไป)

1) Authentication

- Login ผ่าน **Google OAuth** หรือ **Firebase**
- หลัง login ต้องได้ `user_id` ที่ใช้ผูกกับการจอง

2) Seat Map (Real-time)

- แสดงฝั่งที่นั่งของรอบฉาย
- สถานะที่นั่ง:
 - AVAILABLE
 - LOCKED (มีคนเลือกอยู่)
 - BOOKED
- เมื่อมีผู้ใช้เลือกที่นั่ง:
 - ผู้ใช้คนอื่นต้องเห็นการเปลี่ยนแปลงแบบ **Real-time**

✓ แนะนำใช้ WebSocket

3) Booking Flow (หัวใจของโจทย์)

Step Flow

1. ผู้ใช้เลือกที่นั่ง
2. Backend ต้อง:
 - ใช้ **Redis Distributed Lock**
 - Lock ที่นั่งนั้นเป็นเวลา **5 นาที**
3. ระหว่าง lock:
 - ผู้ใช้คนอื่นไม่สามารถเลือกที่นั่งเดียวกันได้
4. หาก:
 - ❌ ไม่ชำระเงินใน 5 นาที → ปลด lock
 - ✅ ชำระเงินสำเร็จ → เปลี่ยนเป็น BOOKED

! ต้องอธิบายเหตุการณ์การออกแบบ Lock Strategy ใน README

2.2 Admin Side

1) Admin Dashboard

- รายการ Booking ทั้งหมด
 - Filter อย่างน้อย 1 อย่าง (เช่น by movie / date / user)
-

2) Audit Logs

ต้องบันทึก Log เหตุการณ์สำคัญ เช่น

- Booking Success
- Booking Timeout
- Seat Released
- System Error (เช่น Lock fail)

สามารถเก็บใน MongoDB หรือส่งผ่าน Message Queue ก็ได้

3. Message Queue Requirement

ต้องมี อย่างน้อย 1 use case จริง เช่น

- ส่ง Event เมื่อ Booking สำเร็จ
- Trigger Notification (mock ได้)
- Async Logging

✗ ไม่รับ MQ ที่มีไว้เฉย ๆ โดยไม่ถูกใช้งาน

4. Non-Functional Requirements

4.1 Concurrency

- ต้องรองรับกรณี:
 - ผู้ใช้หลายคนกดจองที่นั่งเดียวกันพร้อมกัน
 - ต้องไม่มี Double Booking
-

4.2 Security

- แยก Role:
 - USER
 - ADMIN
 - Admin API ต้องไม่ถูกเรียกโดย User
-

4.3 Code Quality

- Structure ชัดเจน
 - Naming สื่อความหมาย
 - ไม่ hardcode config (ใช้ env)
-

5. Deliverables (สิ่งที่ต้องส่ง)

ผู้สมัครต้องส่ง Git Repository ที่มี:

1 Source Code

- Backend
- Frontend

- docker-compose.yml
-

2 README.md (สำคัญมาก)

README ต้องมีหัวข้ออย่างน้อย:

1. **System Architecture Diagram** (วาดง่าย ๆ ก็ได้)
2. Tech Stack Overview
3. Booking Flow อธิบายทีละ Step
4. Redis Lock Strategy
5. Message Queue ใช้ทำอะไร
6. วิธีรันระบบ
7. Assumptions & Trade-offs

📌 README เป็นตัวชี้วัดระดับ Senior/Junior ชัดเจนมาก

3 Optional (ได้คะแนนเพิ่ม)

- Postman Collection
 - Simple Test Case
 - Notification (Email / Line / Mock)
-

6. Evaluation Criteria (ใช้จริงตอนตรวจ)

หมวด	น้ำหนัก
Architecture & Concurrency Design	35%
Correctness (Booking / Lock / Realtime)	30%
DevOps & Security	20%
Code Quality & README	15%

7. สิ่ง “ไม่” คาดหวัง

- UI สวยระดับ Production
- Payment Gateway จริง
- Feature ครบเหมือนระบบโรงแรมจริง

🎯 เราโฟกัส “ความคิด + ความถูกต้อง” มากกว่า Feature เยอะ
